المدرسة الوطنية للعلوم التطبيقية - بني ملال

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵏⵏⴰⵎⵓⵙ ⵏ ⵜⵎⴰⵙⵙⴰⵏⵉⵏ ⵜⵓⵏⵉⵏⵉⵏ - ⴱⵏⵉ ⵎⵍⴰⵍ

Ecole Nationale des Sciences Appliquées - Béni Mellal

RAPPORT TP2 Génie Logiciel

Cycle Ingénieur filière IA-Cybersécurité

Nom : **OURO-AGORO Yasmine**

Apogée **: 23013788**

1. Introduction

L'objectif de ce deuxième TP est de faire évoluer l'application monolithique simple vers une architecture **modulaire**. Cette approche vise à définir des frontières nettes entre les différents domaines métier (Produits, Commandes, Clients) afin de faciliter la maintenance et de préparer une éventuelle migration vers des **microservices**.

2. Objectifs Pédagogiques

- Restructurer le code en modules métier distincts.
- Implémenter des interfaces de communication entre modules.
- Appliquer les design patterns **DTO** (Data Transfer Object) et **Mapper**.
- Utiliser la bibliothèque **MapStruct** pour automatiser les conversions d'objets.

3. Architecture Modulaire

L'application est désormais organisée selon une structure de packages par domaine:

- **Module Product :** Gestion complète du catalogue.
- **Module Order :** Gestion des commandes, dépendant des services publics de Product et Customer.
- **Module Customer :** Gestion des clients (indépendant).

4. Implémentation des Couches Techniques

4.1. Introduction des DTOs et de MapStruct

Pour éviter d'exposer directement les entités JPA à l'extérieur, nous avons mis en place des DTOs :

- ProductDTO : Pour l'envoi de données vers le client.
- CreateProductRequest : Pour la réception et la validation des données entrantes.

La conversion automatique entre ces objets et les entités est gérée par **MapStruct** via l'interface ProductMapper, ce qui garantit un code propre et performant.

4.2. Refactorisation de la Couche Service

La couche Service a été isolée derrière une interface (ProductService) afin de découpler la définition des fonctionnalités de leur implémentation (ProductServiceImpl). Cette séparation permet notamment :

- Une meilleure testabilité.
- L'utilisation de DTOs en entrée et en sortie des méthodes de service.

4.3. Communication Inter-Modules

Conformément aux principes de modularité, le module **Order** ne communique jamais directement avec le repository d'un autre module. Il doit obligatoirement passer par les **services publics** (interfaces) des modules Product ou Customer.

5. Synthèse

Dans le cadre de ce TP, les fonctionnalités suivantes ont été abordées :

- Implémentation complète des modules Customer et Order avec leurs propres Mappers et DTOs.
- Vérification de l'existence d'un client avant la création d'une commande.
- Création d'un endpoint pour consulter l'historique des commandes d'un client spécifique.

6. Conclusion

Cette restructuration en monolithe modulaire représente une étape cruciale dans le cycle de vie d'une application d'entreprise. En limitant les dépendances directes et en utilisant des DTOs, nous avons créé une architecture flexible, prête à être découpée en services indépendants si les besoins de montée en charge le justifient.